new node, and update the links for this new node, and the old head (unless the list was empty).

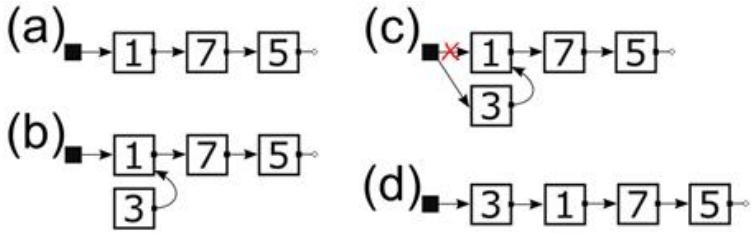


Figure C.6 Insertion at the beginning of a linked list. (A) The original list. (B) Create a new node, whose next pointer is set to the previous head. (C) Correct the head pointer. (D) Final result.

- **At the end**.

  This is not a very practical solution, as we would need to traverse the whole list to find the last node: check figure C.7 to see why. We could think about keeping an extra pointer for the tail of the list, but this would cause an overhead because every method that changes the list would need to check if that reference needs to be updated.

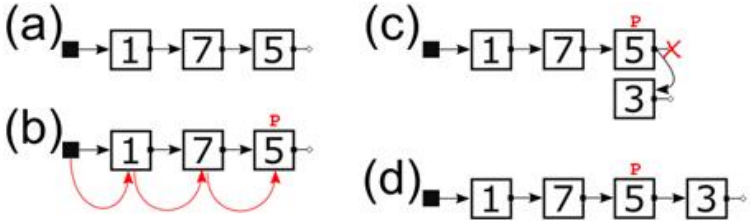


Figure C.7 Insertion at the end of a linked list. (A) The original list. (B) Walk the linked list till the last element: let's call it P. (C) Create a new node and update the next pointer for P. (D) Final result.

- **In any other position**.

  This is useful when keeping the list ordered. However, it's also expensive, requiring linear time in the worst case, as shown in figure C.8.